

浙江大学

ZHEJIANG UNIVERSITY



项目设计报告

SuperKnob 基于 ESP32 的 FOC 多功能智能旋钮

课程名称 嵌入式系统高级实践

小组成员 李彦霖（组长），朱熠正，康鑫杰

学 院 控制科学与工程学院

指导教师 陆玲霞，叶炜

完成日期 2026 年 9 月

目录

摘要	3
1 项目简介	3
1.1 项目背景与目标	3
1.2 系统功能与代码范围	3
1.3 总体工作链路	4
2 硬件平台	4
2.1 核心硬件组成	4
2.2 关键接口与引脚	5
2.3 信号闭环与结构集成	6
3 软件平台	6
3.1 开发环境与依赖	6
3.2 软件分层	6
3.3 两种启动模式	7
4 FOC 原理及其在项目中的应用	7
4.1 磁场定向控制的基本思想	7
4.2 本项目中的控制链路	7
4.3 初始化与运行参数	8
5 四种手感的实现	9
5.1 统一配置结构	9
5.2 棘轮触感	9
5.3 边界触感	9
5.4 阻尼触感	9
5.5 弹簧触感	10
5.6 触感配置与应用映射	10
6 FreeRTOS 双核调度与跨核通信	11
6.1 任务划分	11
6.2 双向消息队列	11
6.3 页面切换的跨核过程	12
6.4 共享状态与并发保护	13
6.5 双核设计的收益	13

7	功能模块	13
7.1	主界面交互	13
7.2	物联网风扇控制	14
7.3	番茄钟与音乐演奏	14
7.4	BLE 电脑控制	15
7.5	钓鱼小游戏	15
7.6	跳一跳	15
8	设计制作调试与结果	16
8.1	项目设计过程	16
8.2	硬件制作与整机装配	17
8.3	软硬件联调过程	17
9	项目总结个人体会与成员分工	17
9.1	项目总结	17
9.2	个人体会	18
9.3	成员分工	18

摘要

SuperKnob 是一套以 ESP32、三相无刷电机、AS5600 磁编码器和 ST7789 显示屏为核心的智能旋钮系统。项目用电机主动输出的可控作用力替代固定机械结构，使同一套硬件能够通过软件切换棘轮、边界、阻尼和弹簧四类触感，并把旋转与触摸输入用于图形界面、电脑控制、计时提醒、音乐演奏、物联网设备控制和小游戏等场景。

系统的软件结构围绕实时控制与图形交互的解耦展开。电机任务固定运行在 ESP32 Core 1，持续读取 AS5600 的角度与速度并执行 SimpleFOC 闭环换相；LVGL 任务固定运行在 Core 0，负责屏幕刷新、页面状态和输入事件。两个任务通过 FreeRTOS 队列传递页面切换、点击反馈、音乐播放和弹簧模式等控制事件，并通过另一条队列返回电机初始化状态。现有代码没有配置相电流采样，因此触感算法给出的“力矩”在实现上是受限的 q 轴电压指令。该实现足以形成稳定可辨识的旋钮触感，同时也明确了后续升级到电流闭环转矩控制的接口边界。

关键词：智能旋钮；磁场定向控制；软件定义触感；FreeRTOS；LVGL；ESP32

1 项目简介

1.1 项目背景与目标

传统旋钮依靠弹簧、钢珠、限位柱等机械结构形成段落感与边界感。一旦机械结构确定，旋钮的档位数量、档距和回弹方式也随之固定。SuperKnob 将无刷电机作为双向力反馈执行器，把“旋钮应该如何用力”写成可切换的控制规律。页面或功能改变时，系统只需修改目标作用力的计算方式，就能让同一个旋钮表现为有清晰吸附点的菜单滚轮、带上下限的参数旋钮、松手回中的弹簧杆，或仅在运动时产生阻力的连续控制器。

项目的工程目标包括三部分。第一，完成 ESP32、BLDC、磁编码器、显示屏、驱动板和触摸输入的整机集成。第二，在 SimpleFOC 的基础上实现稳定的力反馈控制，并把触感参数封装为可切换配置。第三，以 LVGL 页面、BLE HID、局域网风扇控制、计时器、音乐和游戏验证旋钮交互在不同任务中的适配能力。

1.2 系统功能与代码范围

当前固件的主界面包含台灯、番茄钟、音乐、风扇、游戏、跳一跳、电脑、传感器和关于等入口。答辩 PPT 将电脑控制概括为网页滚轮、音量调节和 Alt+Tab 三种模式；现有 `bluetooth_mouse.cpp` 还实现了第四种鼠标左右键模式，因此固件通过触摸 GPIO0 实际循环切换四种 PC 子模式。

报告涉及的主要源文件见表 1。其中，`motor.cpp` 是 FOC 与触感算法的核心；`main.cpp`、`display.cpp` 共同定义双核任务和队列通信；各页面文件负责在具体交互场景中选择触感配置或发送控制事件。

源文件	主要职责
src/main.cpp	创建消息队列，选择主界面或 PC 运行模式，将 LVGL 与 FOC 任务固定到不同 CPU 核
src/motor.cpp	初始化 AS5600、三相驱动和 SimpleFOC，计算棘轮、边界、阻尼与弹簧触感，处理音乐 PWM
src/display.cpp	初始化 TFT 与 LVGL，读取旋钮位置和触摸输入，收发跨任务消息
src/bluetooth_mouse.cpp	将旋钮角度映射为网页滚轮、音量、Alt+Tab 和鼠标左右键输入
src/ui_pages/*.cpp	构建功能页面，并根据页面状态切换 KnobConfig 或发送事件

表 1 项目主要代码模块

1.3 总体工作链路

图 1 表示旋钮的闭环工作链路。AS5600 读取电机轴角度，ESP32 根据页面状态选择触感模型并计算控制量，SimpleFOC 把控制量转换成三相 PWM，电机再把力反馈施加到用户手上。与此同时，LVGL 将同一角度信息解释为菜单焦点变化、参数增减或游戏输入。系统因此形成“物理运动、控制计算、界面状态、触感反馈”相互关联的交互闭环。

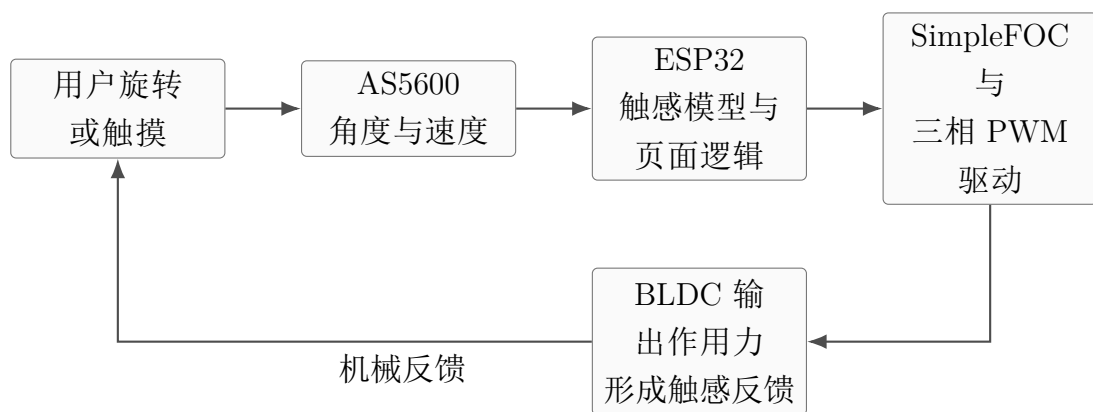


图 1 SuperKnob 的输入、控制与反馈链路

2 硬件平台

2.1 核心硬件组成

主控采用 ESP32 LOLIN32 Lite。该芯片提供双核处理器、FreeRTOS 运行环境、I2C、SPI、PWM、Wi-Fi 与 BLE，能够在单芯片上并行承担图形界面、电机控制和无线通信。执行器为 2804 三相无刷电机，代码按 7 对极建立 BLDCMotor(7) 对象。电机由三

路 PWM 驱动板控制，使能脚由 GPIO12 管理。转子位置由 AS5600 磁编码器提供，固件以 400 kHz I2C 读取传感器。

显示模块使用 240×320 分辨率的 ST7789 TFT，TFT_eSPI 负责 SPI 传输，LVGL 负责页面与控件。显示任务申请一块 240×40 像素的内部 DMA 缓冲区，约占 19.2 KB，通过分条刷新降低图形缓冲对 ESP32 内存的压力。GPIO0 的电容触摸通道同时承担页面确认以及 PC 模式切换输入。

模块	型号或接口	在系统中的作用
主控	ESP32 LOLIN32 Lite	运行 Arduino 框架、FreeRTOS、LVGL、SimpleFOC、Wi-Fi 与 BLE
执行器	2804 BLDC，7 对极	输出双向可控作用力，形成触感与振动反馈
位置反馈	AS5600，I2C 地址 0x36	读取机械角度，供 FOC 和交互逻辑计算位置与速度
电机驱动	三相 3PWM 驱动板	接收 ESP32 的三路 PWM 并驱动 BLDC，相电源按 12 V 配置
显示	ST7789， 240×320	显示主菜单、状态信息与各功能页面
触摸输入	ESP32 GPIO0 电容触摸	作为确认、退出及 PC 子模式切换按键
结构件	自研转接板与外壳	组织屏幕、主控、驱动、编码器及电机的电气和机械连接

表 2 SuperKnob 核心硬件

2.2 关键接口与引脚

表 3 汇总了 motor.cpp 与 platformio.ini 中的实际引脚配置。电机与屏幕分别占用不同的 PWM、I2C 和 SPI 资源，便于在双核任务中并行运行。电机驱动的 voltage_power_supply 设置为 12 V，固件把控制电压上限设为 8 V；初始化时还通过 1000 个递增步骤逐渐放开电压限幅，以减轻对齐完成后的突发作用力。

信号	ESP32 引脚	说明
电机 PWM A/B/C	32 / 33 / 25	三相 3PWM 输出
电机使能	12	高电平使能驱动板
AS5600 SDA/SCL	23 / 5	400 kHz I2C 总线
TFT MOSI/SCLK	19 / 18	ST7789 SPI 数据与时钟
TFT CS/DC/RST/BL	16 / 15 / 13 / 17	片选、数据命令、复位与背光
触摸确认	0	电容触摸通道

表 3 固件中的关键引脚配置

2.3 信号闭环与结构集成

磁铁与电机轴同轴安装后，AS5600 提供连续机械角度。ESP32 将机械角与 7 对极相乘得到电角度，并根据初始化得到的零电角完成换相。三相驱动板把 PWM 占空比转换为绕组电压，电机产生的反作用力通过旋钮轴直接反馈给用户。屏幕、编码器、电机驱动和主控之间的连接由自研转接板重新组织，外壳则负责保证磁铁、编码器、电机轴和显示屏的相对位置。该结构要求电气闭环和机械同轴度同时满足条件，任何安装偏心、磁铁距离不合适或供电压降都会直接表现为触感波动。

3 软件平台

3.1 开发环境与依赖

工程使用 PlatformIO 管理构建和依赖，目标环境为 `lolin32_lite`，底层采用 Arduino framework。`platformio.ini` 固定了主要库版本与 TFT 配置，表 4 列出实际软件栈。项目同时内置 BleCombo 源码，用同一 BLE 连接提供键盘和鼠标 HID 能力。

组件	版本或配置	用途
PlatformIO Espressif32	7.1.1	工程构建、下载与依赖管理
Arduino framework	ESP32 平台默认版本	硬件接口、任务入口与兼容库运行环境
SimpleFOC	2.2.1	传感器角度接入、FOC 换相和 3PWM 输出
LVGL	8.4.0	页面、控件、动画、输入设备与定时器
TFT_eSPI	2.5.0	ST7789 SPI 显示驱动与 DMA 刷新
ESP32 BLE Keyboard	0.3.2 兼容版本	BLE HID 键盘基础能力
项目内置 BleCombo	本地源码	组合键盘与鼠标 HID

表 4 项目软件栈

3.2 软件分层

应用层由各 LVGL 页面和功能控制器组成。页面通过旋转和触摸完成焦点切换、参数调节与状态显示。交互层位于 `display.cpp`，将电机配置中的离散位置变化转换为 LVGL 的 `enc_diff`，并对 GPIO0 触摸进行连续采样和锁存消抖。任务与通信层由 FreeRTOS 任务、队列、软件定时器以及少量临界区构成。控制层位于 `motor.cpp`，负责角度采样、FOC 更新和触感控制量计算。驱动层包括 SimpleFOC、TFT_eSPI、BleCombo、Wi-Fi UDP 与 ESP32 LEDC。

这种分层使页面不直接操作三相 PWM。页面只选择 `KnobConfig` 或发送事件，电机任务在自己的循环中执行配置更新。钓鱼游戏选择配置 5 进入纯粘性阻尼，跳一跳

发送 JUMP_SPRING_START 进入独立弹簧状态机，音乐页面发送 MUSIC_PLAY 暂时把三路 PWM 资源切换到 LEDC 音符输出。

3.3 两种启动模式

固件通过 RTC 非初始化内存中的 `current_os_mode` 区分主界面模式与 PC 模式。硬件复位时该变量被强制置零，系统创建 LVGL 任务并启动主界面；软件重启进入 PC 模式时保留该变量，只直接初始化 TFT 并启动 BLE，从而把较多内存留给 BLE 协议栈。FOC 任务在两种模式中都会启动。PC 模式下，电机任务直接调用 `run_pc_mouse_logic()`，根据旋钮角度同时生成 HID 事件和对应作用力。

4 FOC 原理及其在项目中的应用

4.1 磁场定向控制的基本思想

三相无刷电机需要按照转子磁场方向切换定子电压。FOC 将三相静止坐标系中的电气量变换到随转子旋转的 $d-q$ 坐标系，使励磁方向与产生转矩的方向相互正交。以两相静止坐标为中间量，典型 Park 变换可写为

$$\begin{bmatrix} x_d \\ x_q \end{bmatrix} = \begin{bmatrix} \cos \theta_e & \sin \theta_e \\ -\sin \theta_e & \cos \theta_e \end{bmatrix} \begin{bmatrix} x_\alpha \\ x_\beta \end{bmatrix}, \quad (1)$$

其中 θ_e 是电角度。对于 7 对极电机，电角度在忽略零点偏置时满足 $\theta_e = 7\theta_m$ ， θ_m 为 AS5600 读取的机械角。控制器通常把 d 轴指令设为零，把 q 轴作为主要转矩方向，再通过逆 Park 变换和空间矢量 PWM 生成三相占空比。这样可以避免简单六步换相带来的明显转矩脉动，使旋钮在低速和静止附近也能输出方向连续的作用力。

4.2 本项目中的控制链路

本项目把 SimpleFOC 作为“触感执行层”。上层触感算法根据角度误差、角速度、页面边界或游戏状态计算控制量 u_h ，SimpleFOC 则根据当前电角度把 u_h 施加到合适的 q 轴方向，并用 Space Vector PWM 驱动三相桥。控制链路可表示为图 2。

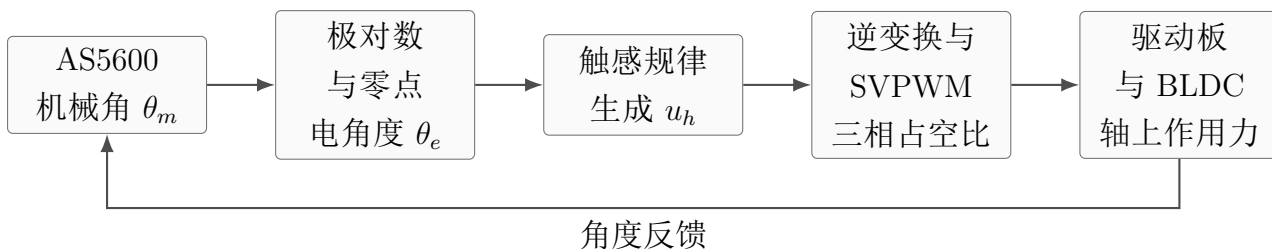


图 2 FOC 在 SuperKnob 中的执行链路

代码首先在 23/5 引脚上启动 400 kHz I2C，将 AS5600 连接到电机对象；随后设置驱动电源为 12 V、传感器对齐电压为 3 V，并选择 Space Vector PWM。电机初始化与 FOC 对齐完成后，控制模式从角度控制切换为 `MotionControlType::torque`。FOC 任务在循环中持续调用 `motor.loopFOC()`，再调用 `motor.move(control)` 写入本次触感控制量。

Listing 1 FOC 初始化的核心代码结构

```
1 I2Cone.begin(23, 5, 400000UL);
2 sensor.init(&I2Cone);
3 motor.linkSensor(&sensor);
4
5 driver.voltage_power_supply = 12;
6 driver.init();
7 motor.linkDriver(&driver);
8
9 motor.voltage_sensor_align = 3;
10 motor.foc_modulation = FOCModulationType::SpaceVectorPWM;
11 motor.init();
12 motor.initFOC();
13
14 motor.controller = MotionControlType::torque;
15 motor.loopFOC();
16 motor.move(control_value);
```

4.3 初始化与运行参数

初始化阶段使用速度 PID 参数 $P = 0.08$ 、 $I = 4$ 、 $D = 0.0002$ ，速度低通时间常数为 0.02 s，角度环比例系数为 20，速度上限为 5 rad/s。完成对齐后，`init_angle()` 在保持当前角度的同时把控制电压限幅从零逐步提升至 8 V。进入触感循环后，速度 PID 对象被复用于位置误差到控制量的映射：不同档距会改变微分项，不同吸附强度会改变比例项与限幅。

电机任务每轮末尾执行 `vTaskDelay(1)`。它没有建立固定周期的硬实时定时器，因此实际更新间隔会受 FreeRTOS tick、串口打印和分支路径影响。对当前交互频率而言，该循环足以提供连续触感；若后续需要可量化的带宽与稳定裕度，应使用固定周期调度并记录实际采样间隔。

5 四种手感的实现

5.1 统一配置结构

常规触感由 KnobConfig 描述，字段包括位置数量、当前位置、单格角宽、吸附强度、边界强度、跨格阈值和阻尼强度。页面通过 `update_motor_config(index)` 复制预设配置，随后发送 CHECKOUT_PAGE，电机任务将当前轴角设置为新的吸附中心并刷新控制参数。该机制把“页面选择何种触感”和“电机如何连续计算作用力”分离开来。

5.2 棘轮触感

棘轮模式维护当前吸附中心 θ_c 。轴角与中心的偏差为

$$e_\theta = \theta_m - \theta_c. \quad (2)$$

当 e_θ 超过“档距 $\times$ 跨格阈值”时，程序把 θ_c 平移一个档距，并同步增加或减少逻辑位置。未跨格时，控制器持续把旋钮拉回最近吸附中心。中心附近设置一个不超过 1° 的死区，减小静止抖动；旋钮静止 500 ms 且仍位于中心附近时，中心以很小的系数缓慢跟随实测角度，用于消除长期偏置。

主菜单使用配置 1：档距约 8.226° ，吸附强度 2.3，位置数量为 0 表示不限制档位。风扇详情页沿用同一物理档距和强度，在页面软件侧把输入灵敏度放大 2 倍，避免通过缩小闭环档距来牺牲稳定性。

5.3 边界触感

边界模式在有限位置范围的首端或末端检测越界方向。如果用户继续向范围外旋转，程序把速度 PID 的比例系数从吸附强度对应值切换为边界强度对应值，同时把输出限幅从 3 提高到 10，从而产生明显的反向作用力。配置 2 和配置 3 都定义了 256 个位置、 1° 档距和起始位置 127；前者带吸附档位，后者关闭档位但保留边界。配置 4 则定义两个相隔 60° 的位置，形成开关式触感。

边界力不是独立的机械挡块，而是有限位置状态与增强控制增益共同形成的软件限位。断电时该限位消失，因此它适合交互提示和参数保护，不应承担人身或结构安全限位功能。

5.4 阻尼触感

阻尼模式不设置目标位置，只对转速施加反向控制量：

$$u_h = -b\hat{\omega}. \quad (3)$$

其中 $b = 0.6$, $\hat{\omega}$ 是低通后的轴速度。代码采用指数滑动平均

$$\hat{\omega}_k = (1 - \alpha)\hat{\omega}_{k-1} + \alpha\omega_k, \quad \alpha = 0.05, \quad (4)$$

并在 $|\hat{\omega}| \leq 0.15$ rad/s 时输出零, 超过死区后把控制量限制在 ± 3 。低通与死区用于抑制 AS5600 量化差分造成的速度尖峰。该模式只有速度项, 没有位置恢复项, 因此旋转时感觉“发沉”, 松手后停在当前位置而不会回到初始点。钓鱼游戏在进入实际游戏阶段时选择配置 5 使用该手感。

5.5 弹簧触感

跳一跳采用独立工作模式而不是普通 KnobConfig。进入游戏时, FOC 任务记录中心角 θ_0 , 令 $\Delta\theta = \theta_m - \theta_0$, 并计算

$$u_h = -k\Delta\theta - b\omega, \quad k = 3.2, \quad b = 0.08. \quad (5)$$

控制量限制在 ± 5.5 。当旋转角超过 100° 时, 程序叠加额外反向项, 使末端逐渐变硬。蓄力比例由 $|\Delta\theta|/100^\circ$ 限制到 0-1 得到。峰值达到 8° 后, 如果角度从峰值回落至少 3° 且速度方向指向中心, FOC 任务就把一次“释放”事件写入共享状态, LVGL 游戏任务读取蓄力比例并开始跳跃动画。

PC 模式中的 Alt+Tab 也采用回中型控制, 但实现更简化: 控制量为 $u_h = -12\Delta\theta$, 越过约 0.55 rad 后触发窗口切换, 回到 0.5 rad 内释放 Alt 键。另一个鼠标左右键模式使用 $u_h = -15\Delta\theta$, 向两侧越过阈值分别按下左右键。

5.6 触感配置与应用映射

索引	位置范围	档距	主要参数	代码中的用途
0	无限制	10°	无吸附	进入 PC 系统前的过渡配置
1	无限制	8.226°	吸附强度 2.3	主菜单、游戏菜单、风扇菜单
2	256 档	1°	吸附 1, 边界 1	台灯、番茄钟等精细离散调节
3	256 档	1°	无吸附, 边界 1	有限范围的连续调节预设
4	2 档	60°	跨格阈值 0.55	开关式强段落预设
5	无限制	不适用	阻尼 0.6	钓鱼游戏的连续追踪
6	无限制	8.226°	吸附 2.3	风扇详情页, 软件侧 2 倍灵敏度

表 5 super_knob_configs 预设与应用关系

6 FreeRTOS 双核调度与跨核通信

6.1 任务划分

主界面模式创建两个固定核任务。Task_lvgl 的栈大小为 4096 字节、优先级为 3，固定运行在 Core 0；Task_foc 的栈大小同为 4096 字节、优先级为 2，固定运行在 Core 1。Arduino loop() 只每 10 s 延时一次，不承担实时业务。页面刷新、动画、输入设备和开机进度均在 LVGL 任务中处理；传感器读取、loopFOC()、触感状态机和电机 PWM 更新均在 FOC 任务中处理。

PC 模式为了给 BLE 栈保留内存，不创建 LVGL 任务，只直接在 TFT 上显示模式说明。FOC 任务仍在 Core 1 运行，并在每个循环中执行 BLE HID 角度映射。由此可见，“双核 LVGL + FOC”是主界面模式的结构，PC 模式则退化为电机任务主导的单业务核结构。

Listing 2 任务与队列创建的核心结构

```
1 motor_msg_Queue = xQueueCreate(10, sizeof(struct _knob_message *));
2 motor_rcv_Queue = xQueueCreate(10, sizeof(struct _knob_message *));
3
4 xTaskCreatePinnedToCore(Task_lvgl, "Task_lvgl", 4096, NULL, 3,
5                          &Task_lvgl_Handle, 0);
6 xTaskCreatePinnedToCore(Task_foc, "Task_foc", 4096, NULL, 2,
7                          &Task_foc_Handle, 1);
```

6.2 双向消息队列

系统创建两条长度为 10 的队列，队列元素均为 _knob_message* 指针。motor_rcv_Queue 从 UI 指向电机，传递页面或功能事件；motor_msg_Queue 从电机指向 UI，传递初始化状态。表 6 给出了代码中已定义的消息协议。

方向	消息	电机或界面侧处理
UI 到 Motor	CHECKOUT_PAGE	以当前轴角重置吸附中心，更新触感参数并产生短振动
UI 到 Motor	BUTTON_CLICK	执行正负方向各 2 个短周期的点击振动
UI 到 Motor	MUSIC_PLAY	退出常规触感，重配三路 LEDC 并开始播放旋律
UI 到 Motor	MUSIC_STOP	停止音乐并通过重启恢复 FOC PWM 资源
UI 到 Motor	JUMP_SPRING_START	记录当前中心角，清空蓄力与释放状态，进入弹簧模式
Motor 到 UI	MOTOR_INIT	将开机进度条更新到 30%
Motor 到 UI	MOTOR_INIT_SUCCESS	FOC 对齐完成后将进度条更新到 75%
Motor 到 UI	MOTOR_INIT_END	电压渐入完成后更新到 100%，随后进入主界面
Motor 到 UI	MOTOR_MUSIC_END	界面保留对应处理分支，当前音乐流程实际通过重启返回

表 6 双向队列消息协议

6.3 页面切换的跨核过程

以从主菜单进入台灯页面为例，LVGL 的点击回调先创建目标页面并执行切屏动画，再调用 `update_motor_config(2)` 选择 1° 精细档位，最后调用 `update_page_status(CHECKOUT_PAGE)`。该函数把消息指针压入 `motor_rcv_Queue`。Core 1 上的 FOC 任务以非阻塞方式接收消息，收到后把当前 `shaft_angle` 设为新的档位中心，按新配置计算 PID 参数，并执行一次短振动。消息处理结束后，FOC 任务立即回到连续 `loopFOC()` 循环。

电机初始化状态沿反方向传递。FOC 任务在传感器初始化前、`initFOC()` 完成后以及电压渐入完成后分别发送三个状态。LVGL 任务最多等待 1 tick 接收消息，并据此更新欢迎页进度条。这样界面不需要轮询底层初始化细节，电机任务也不需要直接访问 LVGL 控件。

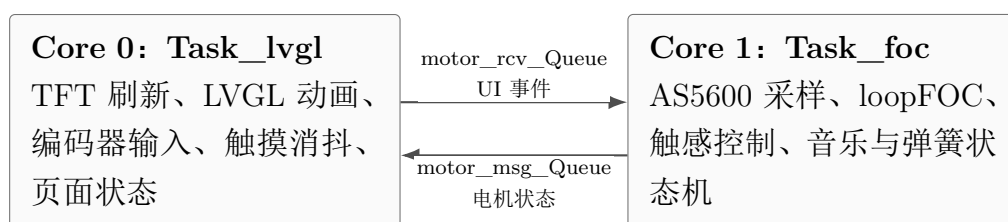


图 3 主界面模式下的双核任务与队列

6.4 共享状态与并发保护

跳一跳的蓄力比例和释放事件没有放入通用队列，而是由 FOC 任务写入三个共享变量，LVGL 任务通过 `get_jump_charge_percent()` 和 `consume_jump_release()` 读取。代码使用 ESP32 的 `portMUX_TYPE` 临界区保护这些变量，避免浮点值或标志在跨核读写时出现不一致。

通用消息队列传递的是指向全局 `LVGL_MSG` 与 `MOTOR_MSG` 的指针，而不是消息结构体副本。当前事件频率较低，发送后通常会很快被另一核取走；但如果同一方向连续发送多个事件，后一次写入可能在前一次消费前覆盖全局对象。更稳健的实现可以把队列元素大小改为 `sizeof(_knob_message)` 并发送结构体值，或为每条消息使用独立存储。两处 `xQueueSend` 的等待时间均为 0，队列满时消息会直接丢失，因此后续还应检查返回值并记录溢出。

6.5 双核设计的收益

FOC 控制对更新连续性敏感，TFT 刷新和页面动画则可能一次传输较多像素。将两者固定到不同 CPU 核后，显示操作不会直接占用电机任务的执行上下文。队列只传递离散事件与少量状态，页面无需了解 FOC 内部变量，电机任务也无需操作 LVGL 对象。该结构降低了模块耦合，并为局域网控制、BLE 和游戏逻辑继续扩展保留了清晰边界。

7 功能模块

7.1 主界面交互

7.1.1 从电机角度到界面焦点

主界面的关键不是单独实现一套滚动算法，而是把电机形成的物理档位直接解释为 LVGL 的焦点移动。FOC 任务持续读取轴角 θ_m ，并维护当前档位中心 θ_c 。主界面采用配置 1，档距约为 8.23° ，档位数量设为 0，表示菜单可以连续滚动而不设置机械意义上的首尾限位。当角度偏差

$$e_\theta = \theta_m - \theta_c \quad (6)$$

超过档距与跨格阈值的乘积时，电机任务将 θ_c 平移一个档距，同时把逻辑位置变量 `position` 加一或减一。这样，连续角度先被触感状态机量化为稳定的整数档位；手停在吸附点附近时，位置值不会因传感器细小波动反复变化。

显示任务把旋钮注册为 `LV_INDEV_TYPE_ENCODER`。输入回调 `encoder_read()` 比较本次与上次的档位值：位置增大时输出 `enc_diff=+1`，位置减小时输出 `enc_diff=-1`。编码器输入与默认 `lv_group` 绑定后，LVGL 按差分方向把焦点移动到前一个或后一个可交互按钮。因此，菜单选择形成了“轴角跨过档位阈值—档位整数变化—编码器差分

一焦点切换”的逐级映射。第 6 章已经说明了 FOC 与 LVGL 分核运行及消息传递，本节关注的是这条映射如何把物理旋转转化为用户可见的选择结果。

7.1.2 焦点的视觉表达与确认操作

菜单容器使用纵向 Flex 布局，并开启纵向滚动和中心吸附。焦点变化后，目标按钮自动滚动到圆屏中央。为了让列表与圆形屏幕匹配，`scroll_event_cb()` 根据子项中心与容器中心的纵向距离 d_y 计算横向偏移

$$x = r - \sqrt{r^2 - d_y^2}, \quad |d_y| < r, \quad (7)$$

超出半径时则把 x 限制为 r 。离中心越远的条目横向缩进越大、透明度越低，于是列表在视觉上呈现沿圆弧远离观察者的效果；位于中心的条目偏移最小且完全不透明，自然成为当前焦点。界面没有额外绘制固定光标，焦点位置、中心吸附和电机档位三者共同给出选择反馈。

GPIO0 的触摸输入在连续检测到有效触摸后向 LVGL 报告按下状态，当前焦点按钮随后产生 `LV_EVENT_CLICKED`。按钮回调负责创建目标页面、播放切屏动画，并按功能需要切换电机配置或工作模式。例如进入番茄钟时换为精细档位，进入音乐时发送播放事件，进入跳一跳时由游戏页面启动弹簧模式。旋转负责“选择”，按下负责“确认”，而每跨一格都能感到真实吸附，使视觉焦点与手上感知保持一致。

7.2 物联网风扇控制

本节具体内容待补充

7.3 番茄钟与音乐演奏

7.3.1 番茄钟的设计思路与核心功能

番茄钟把计时与触觉提醒分开实现。页面提供工作、休息和退出三个操作，工作时间为 25 min，休息时间按演示配置为 2 min；FreeRTOS 软件定时器每秒更新一次 `MM:SS`，倒计时结束后启动提醒定时器。LVGL 仅负责按钮事件和时间显示。

进入页面时采用配置 2，即 256 个位置、 1° 档距的精细档位，便于移动焦点。按钮确认和超时提醒均通过 `BUTTON_CLICK` 触发 `motor_shake()`，使电机产生短促振动；退出时停止定时器，并恢复主界面的连续档位配置。因此电机在操作阶段提供细腻的档位感，在计时结束后兼作触觉提醒器。

7.3.2 电机发声原理

无刷电机的绕组电流会产生磁场，并与永磁转子相互作用形成电磁转矩和径向力。当驱动电压按音频频率周期变化时，电磁力会激励转子、定子和外壳产生微小机械振动，结构表面推动空气形成声音。信号频率决定音高，持续时间决定节奏，电压、绕组阻抗和机械共振则影响响度与音色。

正常 FOC 根据转子角度合成旋转磁场，以获得平滑作用力；音乐模式则暂时停止 `loopFOC()`，主动向绕组施加周期激励，把机械振动作为主要输出。由于电机没有扬声器纸盆，频率响应也不平坦，因此其效果更接近蜂鸣器，旋律能够辨认但音色较为粗糙。

7.3.3 音乐模式的实现

音乐页面通过 `MUSIC_PLAY` 通知电机任务接管播放。程序先令 FOC 输出为零，再把三个相线 PWM 引脚切换到 ESP32 LEDC 通道。旋律与时值保存在两个数组中，播放逻辑使用 `millis()` 非阻塞地推进音符；休止符输出零频率，其余音符按

$$f_{\text{out}} = 3f_{\text{note}} \quad (8)$$

提高到 3 倍后交给 `ledcWriteTone()`，从而按音符频率和时值组织旋律。

三个相线由 LEDC 产生随音符切换的周期性 PWM，使绕组电流形成相应频率的电磁激励，并通过电机结构振动输出可辨识的旋律。播放结束或触摸退出时，程序关闭音调与驱动并重启 ESP32，以重新初始化 SimpleFOC 和恢复旋钮手感。

7.4 BLE 电脑控制

本节具体内容待补充

7.5 钓鱼小游戏

本节具体内容待补充

7.6 跳一跳

7.6.1 把旋钮设计成蓄力弹簧

跳一跳没有沿用普通 `KnobConfig` 档位，而是在电机任务中设置独立的 `MOTOR_MODE_JUMP`。开始一局时记录当前轴角为中心 θ_0 ，旋转量为 $\Delta\theta = \theta_m - \theta_0$ ，

电机输出采用弹簧与阻尼组合：

$$u_h = -3.2\Delta\theta - 0.08\omega. \quad (9)$$

比例项形成越拧越紧的回中力，速度项抑制回弹振荡；输出限制在 ± 5.5 ，超过 100° 后进一步增强反向作用力。蓄力比例为

$$c = \text{clip}\left(\frac{|\Delta\theta|}{100^\circ}, 0, 1\right) \quad (10)$$

当峰值达到 8° 、角度回落至少 3° 且旋钮正向中心运动时，程序判定用户松手。FOC 任务把蓄力值和释放标志写入临界区保护的共享状态，LVGL 任务只读取结果。

7.6.2 游戏状态与得分规则

游戏定时器每 20 ms 更新蓄力条并检查释放事件。释放后，小方块沿抛物线起跳，距离、高度和动画时长均随蓄力比例增大：

$$x_{\text{end}} = x_{\text{start}} + 175c, \quad (11)$$

$$h = 48 + 48c, \quad (12)$$

$$T = 500 + 280c \text{ ms} \quad (13)$$

目标平台的位置和宽度随机生成。落在平台上可得分，落入中心区域还能获得连续加分；成功后生成下一平台，落空则进入下落和结束状态。该模块形成“旋转蓄力—电机回弹—释放检测—跳跃与落点反馈”的闭环，LVGL 只承担蓄力条、平台、方块和得分绘制，而弹簧手感本身就是游戏规则的一部分。

8 设计制作调试与结果

8.1 项目设计过程

本项目的设计过程主要经历了方案选型、原理学习、功能验证、整机联调和结构完善五个阶段。项目初期，我们希望通过制作一套实际装置学习磁场定向控制（FOC）算法，最初考虑以平衡车为实现目标。经过资料调研与方案比较后，我们认为智能旋钮能够在较小的硬件规模下直观呈现位置闭环、转矩控制和触觉反馈，更适合作为 FOC 的学习与实践载体，因此最终确定以多功能智能旋钮为项目方向，并参考了 [SmartKnob 开源项目](#) 的软件可配置限位与虚拟档位设计思路。

在原理学习与功能验证阶段，我们采购了 DengFOC-V3P 开发板及配套电机套件，结合教学课程学习坐标变换、位置检测、电流控制和闭环调节等 FOC 核心原理。随后在 PlatformIO 环境中逐步完成电机驱动与参数调试，并分别实现连续旋转、档位吸附、

限位和回弹等触感效果。通过这一阶段的实践，我们进一步熟悉了 ESP32、磁编码器、电机驱动套件及嵌入式开发工具链的使用方法，也为后续人机交互功能奠定了基础。

进入整机设计阶段后，为满足图形界面显示需求，系统加入了 ST7789 显示屏并采用 LVGL 构建交互页面。由于 DengFOC-V3P 开发板可用引脚数量有限，电机驱动、角度传感器、显示屏和触摸输入还需要分别满足 PWM、I²C、SPI 等接口要求，直接接线不仅繁杂，也不利于稳定调试。为解决这一问题，我们自主设计了转接板，对电源与信号接口进行集中引出和重新分配，并完成焊接、导通检查与上电测试，从而提高了整机连接的可靠性。

硬件基础稳定后，项目进入软硬件联调阶段。我们利用 ESP32 的 FreeRTOS 双核与任务调度能力，将电机控制和 LVGL 图形显示分别组织为独立任务，通过共享状态与消息实现旋钮触感、界面焦点和页面切换之间的协同。完成主界面交互框架后，小组成员分别负责不同功能页面的开发，再统一进行集成、稳定性测试和代码整理。

最后，为提升设备的完整性与外观效果，我们使用 SolidWorks 对 SuperKnob 外壳及内部安装结构进行建模，并结合显示屏、电机、转接板和连接线的实际尺寸反复调整结构。完成零件制作与整机组装后，项目形成了集电机触觉反馈、图形界面和多种应用功能于一体的多功能智能旋钮。

8.2 硬件制作与整机装配

本节具体内容待补充

8.3 软硬件联调过程

本节具体内容待补充

9 项目总结个人体会与成员分工

9.1 项目总结

本节具体内容待补充

9.2 个人体会

9.2.1 李彦霖

作为项目组长，我参与了项目从方案构想到整机实现的全过程。从零搭建项目架构时，我需要统筹电机控制、LVGL 界面和各功能模块之间的关系，并逐步确定任务划分、通信方式与开发顺序。这一过程使我深入理解了 FOC 的基本原理与触感控制方法，也掌握了利用 FreeRTOS 进行任务调度和跨任务通信的思路。

在开发过程中，我进一步锻炼了软硬件联合调试能力，学会从接线、驱动、控制参数和程序逻辑等多个方面定位问题。同时，组长工作也让我认识到及时沟通和明确分工的重要性。通过与队友协调开发进度、讨论方案并共同解决问题，我在技术实践和团队协作方面都获得了较大提升。

9.2.2 朱熠正

本节具体内容待补充

9.2.3 康鑫杰

本节具体内容待补充

9.3 成员分工

成员	具体分工
李彦霖	FOC 触感引擎实现，LVGL + 电机联调，音乐功能与电机交互展示，跳一跳游戏设计
朱熠正	触摸引脚调试，BLE 电脑控制 + 番茄钟，物联网功能开发
康鑫杰	转接板设计 + 外壳建模，整机装配与集成调试，钓鱼游戏设计
三人共同工作	FOC 原理学习、整机联调、稳定性测试、代码整理与结题汇报

表 7 成员分工记录